

EXPERIMENT 02: RELATIONAL SCHEMA AND REFERENTIAL INTEGRITY

AIM

To convert an ER diagram into a relational schema and implement the schema using MySQL CREATE TABLE statements. To demonstrate Primary Key, Foreign Key, NOT NULL, UNIQUE, ON DELETE CASCADE, and ON DELETE SET NULL constraints by inserting sample data and testing referential integrity.

THEORY

A relational schema represents a database structure using tables, columns, and relationships. Each table contains records, and relationships between tables are established using primary keys and foreign keys.

Primary Key: Uniquely identifies each record in a table.

Foreign Key: Establishes a relationship between two tables and maintains referential integrity.

NOT NULL: Prevents a column from storing NULL values.

UNIQUE: Ensures that values in a column are not duplicated.

ON DELETE CASCADE: Automatically deletes related records when the referenced parent record is deleted.

ON DELETE SET NULL: Sets the foreign key value to NULL when the referenced parent record is deleted. The foreign key column must allow NULL values.

RELATIONAL SCHEMA

1. Department

- Dept_id — Primary Key
- Dept_name — NOT NULL and UNIQUE

2. Student

- Student_id — Primary Key
- Name — NOT NULL
- Email — UNIQUE
- Dept_id — Foreign Key referencing Department(Dept_id)

3. Course

- Course_id — Primary Key
- Course_name — NOT NULL
- Credits — NOT NULL

4. Enrollment

- Student_id — Foreign Key referencing Student(Student_id)
- Course_id — Foreign Key referencing Course(Course_id)
- Enroll_date — NOT NULL
- Composite Primary Key: (Student_id, Course_id)

SQL CODE

Step 1: Create the Database

```
CREATE DATABASE College;
```

USE College;

Step 2: Create the Department Table

```
CREATE TABLE Department (  
    Dept_id INT PRIMARY KEY,  
    Dept_name VARCHAR(50) NOT NULL UNIQUE  
);
```

Step 3: Create the Student Table

```
CREATE TABLE Student (  
    Student_id INT PRIMARY KEY,  
    Name VARCHAR(50) NOT NULL,  
    Email VARCHAR(100) UNIQUE,  
    Dept_id INT,  
    FOREIGN KEY (Dept_id)  
        REFERENCES Department(Dept_id)  
        ON DELETE SET NULL  
);
```

Step 4: Create the Course Table

```
CREATE TABLE Course (  
    Course_id INT PRIMARY KEY,  
    Course_name VARCHAR(100) NOT NULL,  
    Credits INT NOT NULL  
);
```

Step 5: Create the Enrollment Table

```
CREATE TABLE Enrollment (  
    Student_id INT,  
    Course_id INT,  
    Enroll_date DATE NOT NULL,  
    PRIMARY KEY (Student_id, Course_id),  
    FOREIGN KEY (Student_id)  
        REFERENCES Student(Student_id)  
        ON DELETE CASCADE,  
    FOREIGN KEY (Course_id)  
        REFERENCES Course(Course_id)  
        ON DELETE CASCADE
```

);

INSERT SAMPLE DATA

INSERT INTO Department VALUES

(101, 'Computer Science'),

(102, 'Information Technology'),

(103, 'Electronics');

INSERT INTO Student VALUES

(1, 'Rahul Sharma', 'rahul@example.com', 101),

(2, 'Priya Singh', 'priya@example.com', 102),

(3, 'Aman Kumar', 'aman@example.com', 101);

INSERT INTO Course VALUES

(201, 'Database Management Systems', 4),

(202, 'Computer Networks', 3),

(203, 'Operating Systems', 4);

INSERT INTO Enrollment VALUES

(1, 201, '2026-09-01'),

(1, 202, '2026-09-02'),

(2, 201, '2026-09-03'),

(3, 203, '2026-09-04');

DISPLAY TABLE DATA

SELECT * FROM Department;

SELECT * FROM Student;

SELECT * FROM Course;

SELECT * FROM Enrollment;

DEMONSTRATION OF REFERENTIAL INTEGRITY

1. Test ON DELETE SET NULL

DELETE FROM Department

WHERE Dept_id = 102;

SELECT * FROM Student;

Expected Outcome: Student Priya Singh remains in the Student table, but her Dept_id becomes NULL because the referenced department has been deleted.

2. Test ON DELETE CASCADE on Student

```
DELETE FROM Student
```

```
WHERE Student_id = 1;
```

```
SELECT * FROM Enrollment;
```

Expected Outcome: Student 1 is deleted, and all enrollment records associated with Student 1 are automatically deleted.

3. Test ON DELETE CASCADE on Course

```
DELETE FROM Course
```

```
WHERE Course_id = 201;
```

```
SELECT * FROM Enrollment;
```

Expected Outcome: Course 201 is deleted, and all enrollment records associated with Course 201 are automatically deleted.

4. Test UNIQUE Constraint

```
INSERT INTO Department
```

```
VALUES (104, 'Computer Science');
```

Expected Outcome: MySQL generates a duplicate-entry error because the department name 'Computer Science' already exists.

5. Test Foreign Key Constraint

```
INSERT INTO Enrollment
```

```
VALUES (99, 202, '2026-09-05');
```

Expected Outcome: MySQL generates a foreign key constraint error because Student_ID 99 does not exist in the Student table.

6. Test Composite Primary Key

```
INSERT INTO Enrollment
```

```
VALUES (2, 202, '2026-09-06');
```

```
INSERT INTO Enrollment
```

```
VALUES (2, 202, '2026-09-07');
```

Expected Outcome: The first statement succeeds if the record does not already exist. The second statement fails because the combination of Student_id and Course_id must be unique.

OUTPUT

byteXL

queries.sql

MySQL

Run

AI Agent

160 ms

```
7  -- Create Tables
8  CREATE TABLE Department (
9      Dept_id INT PRIMARY KEY,
10     Dept_name VARCHAR(50) NOT NULL UNIQUE
11 );
12
13 CREATE TABLE Student (
14     Student_id INT PRIMARY KEY,
15     Name VARCHAR(50) NOT NULL,
16     Email VARCHAR(100) UNIQUE,
17     Dept_id INT,
18     FOREIGN KEY (Dept_id)
19         REFERENCES Department(Dept_id)
20         ON DELETE SET NULL
21 );
22
23 CREATE TABLE Course (
24     Course_id INT PRIMARY KEY,
25     Course_name VARCHAR(100) NOT NULL,
26     Credits INT NOT NULL
27 );
28
29 CREATE TABLE Enrollment (
30     Student_id INT,
31     Course_id INT,
32     Enroll_date DATE NOT NULL,
33     PRIMARY KEY (Student_id, Course_id),
34     FOREIGN KEY (Student_id)
35         REFERENCES Student(Student_id)
36         ON DELETE CASCADE,
37     FOREIGN KEY (Course_id)
38         REFERENCES Course(Course_id)
39         ON DELETE CASCADE
40 );
41
42 -- Insert Sample Data
43 INSERT INTO Department VALUES
44 (1, 'CSE'),
45 (2, 'ECE');
46
47 INSERT INTO Student VALUES
48 (101, 'Abhinav', 'abhinav@gmail.com', 1),
49 (102, 'Rohan', 'rohan@gmail.com', 2);
50
51 INSERT INTO Course VALUES
52 (201, 'DBMS', 4),
53 (202, 'Python', 3);
54
55 INSERT INTO Enrollment VALUES
56 (101, 201, '2026-08-19'),
57 (102, 201, '2026-08-19'),
58 (101, 202, '2026-08-19'),
59 (102, 202, '2026-08-19');
60
61 -- Verify Data
62 SELECT * FROM Student;
63 SELECT * FROM Enrollment;
64 SELECT * FROM Department;
```

STDIN

Input for the program (Optional)

Output:

Student_id	Name	Email	Dept_id
101	Abhinav	abhinav@gmail.com	1
102	Rohan	rohan@gmail.com	2

Student_id	Course_id	Enroll_date
101	201	2026-08-19
101	202	2026-08-19
102	201	2026-08-19
102	202	2026-08-19

Dept_id	Dept_name
1	CSE
2	ECE

Ready

Light t3 MySQL

Feedback

New

History

Save

RESULT

The ER diagram was converted into a relational schema, and the corresponding MySQL tables were created successfully. Primary Key, Foreign Key, NOT NULL, UNIQUE, ON DELETE CASCADE, and ON DELETE SET NULL constraints were implemented. Sample data was inserted, and referential integrity was demonstrated through deletion and constraint-violation tests.